

SPA



Single Page Application

¿Qué es?

- Una metodología de crear un cliente de frontend que maneje dinámicamente el input del usuario
- Eliminamos la necesidad de interactuar con el backend lo máximo posible
- Pasamos de una lógica thin client a un thick client

Ventajas

- No se requiere recarga de la pantalla
 - Más dinamismo
- Separation of concern
 - Podemos crear lógica en el cliente sin necesidad de interactuar con el backend
 - Simplifica más el desarrollo, nos alejamos del mono-repo para así reducir los puntos de falla.

Desventajas

- Mucha más lógica del lado del usuario
 - Hacemos 2 veces todo, ya que tenemos que tener client side validation también
- Requiere descarga del framework antes del first impression
- Si no somos cuidadosos podemos perder performance por varios lados
 - Al bajarse muchos archivos a la vez podemos ser capeados por el browser (descarga paralela)

Modules

JS Modules son unidades de independientes de código reusable.

Require JS (de James Burke, ~2009)

RequireJS es la implementación más popular de AMD (Asynchronous Module Definition). Cargar módulos de forma asíncrona en el navegador, resolviendo dependencias en runtime.

- Implementa AMD — pensado específicamente para el browser
- Carga asíncrona: no bloquea el **render**
- Resuelve dependencias en tiempo de ejecución (runtime)
- Sintaxis `define([...deps], fn) / require([...], cb)`

ES Modules (ES2015 ->)

ESM se estandariza en ES2015 (ES6), spec finalizada en junio de 2015, aunque el soporte nativo en navegadores recién llegó fuerte alrededor de 2017-2018 con `<script type="module">`.

- `import / export` — parte del lenguaje, no una librería
- Estructura estática: `imports/exports` se resuelven en parse time
- Habilita tree-shaking y optimización de bundlers
- Live bindings, strict mode y scope propio por default

JS



```
import { myExport } from "/modules/my-module.js";
```

JS



```
import { foo, bar } from "/modules/my-module.js";
```

JS



```
import myDefault from "/modules/my-module.js";
```

JavaScript

Module

Cheatsheet



Name Export

```
export const name = 'value'
```

Name Import

```
import { name } from '...'
```

Default Export

```
export default 'value'
```

Default Import

```
import anyName from '...'
```

Rename Export

```
export { name as newName }
```

Name Import

```
import { newName } from '...'
```

Export List + Rename

```
export {  
  name1,  
  name2 as newName2  
}
```

Import List + Rename

```
import {  
  name1 as newName1,  
  newName2  
} from '...'
```



Javascript Runtimes

The environment built around an engine (normalmente v8) that adds APIs and features so JavaScript can do more than what browsers provide.

Javascript runtimes



- Node.js
 - El primer runtime
- Deno / Bun
 - Reemplazos que hacen a “tapar huecos” que deja node.js
 - Más rápido, más compatible, más tipado, etc
 - New is always better

Package managers

Herramientas para hostear, descargar organizar paquetes de JavaScript

Managing packages

- Hosting packages (package registry)
 - Self hosting
 - Private hosting
 - Public hosting
 - JSR, NPM, Github, AWS, Etc
- CLI package managers
 - NPM, YARN, PNPM, DENO, BUN

JS Bundler

Herramienta de desarrollo que combina y procesa archivos principalmente JS para que sean consumibles por browsers

Webpack, esbuild, Vite

- Webpack fue por muchos años el principal bundler para procesar los distintos módulos de js, css, html, etc, en un paquete de código final listo para el browser.
- Hoy en día los bundlers actúan como orchestradores de distintas librerías que saben de compilación, transpilación, minificación, tree-shaking y otros procesos optimizar tanto el desarrollo como la performance de la página final.
- Un ejemplo moderno sería Vite, creado por el creador de Vue.js

Component framework

Herramientas de templating de html en lo más básico

React ←

Non opinionated, only view layer

React router

Vite / Next.js

Tailwind css / Shadcn

Remix / Zustand

Jest / Vitest

→ Angular

Heavily opinionated, enterprise style

Angular router

Esbuild

Angular material

RxJs

Karma / Vitest

React Vue.js Angular Preact Svelte Alpine.js
Lit Solid Qwik Stencil HTMX

Mode:

Value

Rank

View:

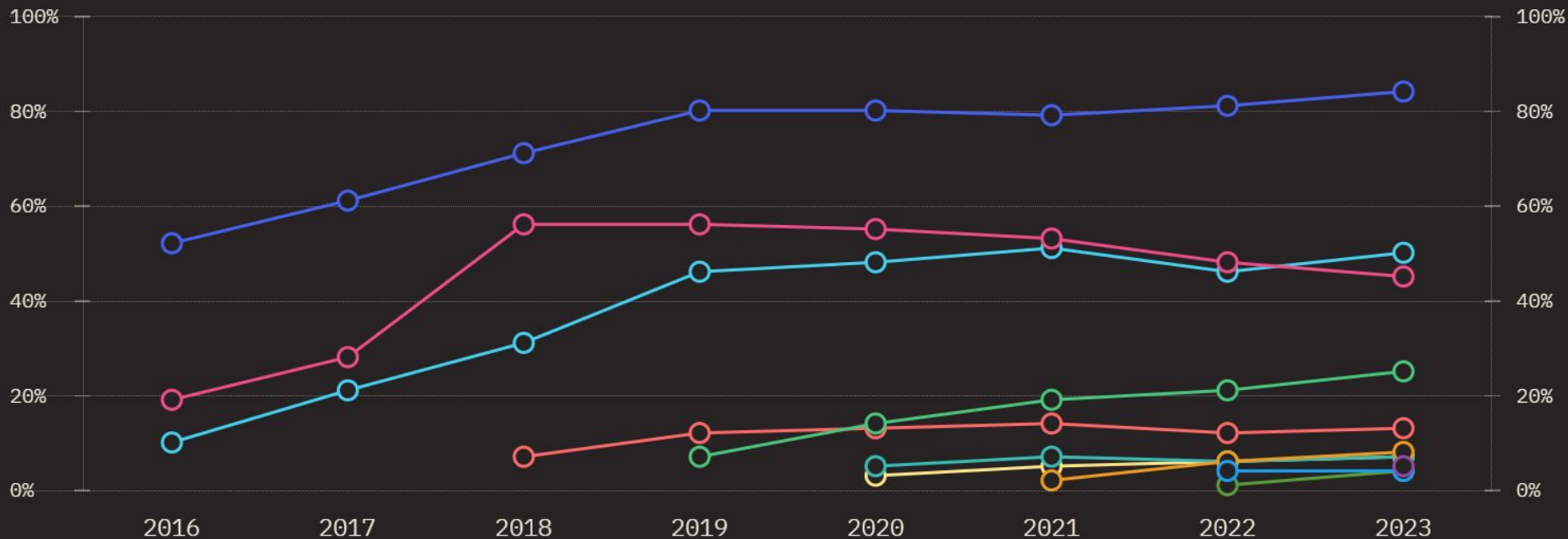
Usage

Awareness

Interest

Retention

Positivity



Routing

- ¿Porqué es relevante la URL?
 - Bookmarks
 - Compartir URLs
 - Refrescar
- Errores 404, 403 deben ser manejados mayormente en el cliente.
- Por más que usamos la URL, todos los redirects y los cambios de vista son internos, Salvo por temas de performance, no se hacen nuevos requests al servidor por más que cambie la URL.

I18n

React: `react-i18next`

Angular: `@angular/localize`

- Pluralización
- Interpolación de variables
- Formatos de fecha/hora/moneda según locale

Testing

React: Jest + React Testing Library.

Angular: Jasmine + Karma o Jest.

Existen varias librerías de testing, no es obligatorio utilizar una en específico.

SPA Hosting

Static hosting (Spring)

Cache

- Condicional vs No-condicional

File revving

- Viene de revisión + versioning.
- Nace para que el cache no condicional funcione
- El file revving se basa en técnicas de hashing, iterando los nombres de los archivos
- Iteramos los nombres de los archivos
 - Archivo Nuevo -> lo bajo

Minificación

- Example: [JavaScript Minifier \(javascript-minifier.com\)](https://javascript-minifier.com)
- Queremos optimizar recursos
- JS al ser ejecutado, es muy verborrágico, innecesariamente.
- A la vez ofuscamos levemente el código
- Técnicas
 - Acortamos variables
 - Obviamos ; cuando se puede
 - Optimizamos funciones
 - Eliminamos comentarios, breaklines, etc.
- También se aplica al css

Autenticación

- Authorization header vs Cookies
 - Local Storage es vulnerable a ataques XSS.
 - Cookies son susceptibles a man-in-the-middle, en especial cuando TODOS los requests envían ambos tokens.
 - Las cookies nos dan menos control de cuando mandarse, sumando así el overhead de red, que impacta a todo, incluidos assets.
- Lógicas posibles
 - Todos los endpoints pueden loggear
 - /login nos devuelve en el body la información de loggeo

Spring + Cache

- Caching no condicional de recursos
- *registry.setCachePeriod()* en los resource handlers
- “cache-control” header en las responses
 - `response.setHeader("Cache-Control", "public, max-age="+ MAX_TIME +", immutable");`
- Se puede agregar un filtro en las configuraciones que haga esto para todas las responses (hay que implementar el filtro)

```
<!-- Unconditional Cache Filter -->
<filter>
  <filter-name>unconditionalCacheFilter</filter-name>
  <filter-class>ar.edu.itba.paw.webapp.config.UnconditionalCacheFilter</filter-class>
</filter>
```

- <https://www.baeldung.com/cachable-static-assets-with-spring-mvc>
- <https://www.baeldung.com/spring-mvc-cache-headers>

Spring + servir la página

- Queremos hacer mvn package y que genere un paquete que tenga todo
- Vamos a usar un plugin que es el front end maven plugin
- github.com/eirslett/frontend-maven-plugin
 - Tiene todo: bower, npm, webpack, etc
 - Es el bridge entre el mundo maven y el mundo javascript
- Hay que tocar la configuración del proyecto webapp para que devuelva todos los recursos estáticos nuevos
- Hay que agregar la dependencia del módulo frontend para que se ejecute primero